Web Technologies Project Report

Index Table

1. Introduction

This report documents the development of a web application for accessing and analyzing anime data. The project was designed to serve anime fans seeking to explore series, characters, and ratings.

The system architecture follows a three-tier microservices approach, composed of two Express.js servers and one Spring Boot server. This design separates responsibilities clearly. The main Express server handles request coordination and web page delivery. The secondary Express server manages dynamic data such as reviews, recommendations, and user ratings stored in MongoDB. The Spring Boot server is responsible for static reference data, including anime details, characters, and people, stored in PostgreSQL.

The web interface was built using Handlebars templating engine with JavaScript and CSS, providing users with an intuitive platform to query and explore the extensive anime dataset.

2. Solution

2.1 Main Express Server

2.1.1 Design and its Motivations

The central server was built using Express.js and acts as the main entry point of the system. Its role is to route requests and coordinate communication between backend services, without directly accessing any database.

The project follows a modular design where each page has its own JavaScript file and controller. This makes the system easier to maintain and allows each part to focus on a specific task. The main advantage of this design is better performance and scalability, since heavy operations are handled by specialized servers

By delegating all database operations to specialized servers, the main server remains lightweight and can handle thousands of concurrent requests without bottlenecks. This addresses the assignment requirement for a fast central server.

Several requests require the combination of dynamic data, such as user-specific favorites stored in MongoDB, with static data including anime, character, and people details provided by the Spring Boot service.

2.1.2 Issues

During development, the main challenges encountered were:

- **Aggregation of data from multiple services:** To build a complete web page, the central server must collect and merge dynamic data provided by the Express-based services with static data retrieved from the Spring Boot server. This aggregation process introduces additional complexity, as it requires careful coordination between services. The challenge

becomes more significant when data is misaligned or when the services exhibit different response times, potentially affecting consistency and user experience.

2.1.3 Requirements

- Quality of the interface: Satisfied by using clear and well-organized views, appropriate UI elements, and avoiding confusing or overloaded pages.
- Systematic use of asynchronous JavaScript communication: Satisfied through consistent use of asynchronous requests (Axios) to communicate with backend services without blocking the interface.
- Correct communication with the Node.js server: Satisfied by reliable interaction between the main Express server and the secondary Express server for dynamic data retrieval.
- Quality of the solution: Satisfied through a modular and well-structured architecture that separates responsibilities and simplifies maintenance.

2.1.4 Limitations

Multiple parallel API calls using Promise.all with .map() can overwhelm backend servers and increase response times.

2.2 Secondary Express Server (MongoDB)

2.2.1 Design and its Motivations

The secondary server was developed using Express.js with Mongoose as the ODM (Object Document Mapper) for MongoDB. This server is responsible for managing the anime dynamic data that changes frequently, including user profiles, favorites, ratings, stats , and recommendations.

The database schema consists of five mongo collections: profiles (user information and watch statistics), favorites (user's favorite anime, characters, and people), ratings (user scores and watch status for anime), stats (aggregated viewing statistics per anime), and recommendations (anime-to-anime recommendations).

2.2.2 Issues

During development, the main problems we fought were:

- **Query Performance**: Initial queries on the ratings and favorites collections were extremely slow (2-3 seconds per request) due to the large dataset size. This was resolved by implementing database indexes on the username field, reducing query times to under 100ms.
- **Data Enrichment Overhead**: User favorites and ratings store only reference IDs, requiring additional calls to the Spring Boot server to retrieve names and images. This was mitigated through parallel requests using Promise.all() and implementing pagination to limit the number of enrichment calls per request.
- **CSV Data Import**: Importing large CSV files (1M+ ratings) required careful handling to avoid memory issues and ensure data integrity during the import process.

2.2.3 Requirements

-Must handle large datasets efficiently: Satisfied through MongoDB indexing and query optimization.
-Must provide RESTful endpoints for all collections: Satisfied with dedicated routes for profiles, favorites, ratings, stats, and recommendations.
-Must support filtering by username and anime ID: Satisfied through parameterized endpoints (e.g., /ratings/user/:username, /stats/:malId).
-API Documentation: Satisfied through Swagger/OpenAPI documentation accessible at /api-docs.

2.2.4 Limitations

-No Real-Time Updates: The current implementation uses static imported data. A production system would require mechanisms for real-time data synchronization with external sources.
-Limited Query Capabilities: The API provides basic filtering by username or anime ID. More complex queries (e.g., filtering ratings by score range or date) are not currently supported.

2.3 Spring Boot Server (PostgreSQL)

2.3.1 Design and its Motivations

The primary server was developed using Spring Boot and PostgreSQL as the relational database. This server is responsible for managing static reference data related to the anime domain, including anime details, characters, staff, voice actors.

PostgreSQL ensures that relationships between anime, characters, and staff are consistent.
Static anime data is separated from dynamic user data, making the system easier to understand and maintain.

2.3.2 Issues

During development, the following issues were encountered:

- **Slow Initial Queries:** Some endpoints were slow when returning data that involved several related tables. This was improved by simplifying queries and returning only the necessary information.

- **Too Much Data Returned:** Early versions returned more data than needed, increasing response size. This was fixed by limiting the fields sent in the response.

2.3.3 Requirements

The server satisfies all assignment specifications, correctly handling REST endpoints, JPA/PostgreSQL persistence, and standard HTTP status codes.

2.3.4 Limitations and Considerations

Lazy-loaded JPA relationships are used for efficiency, loading related entities only when needed. However, this can cause N+1 query problems when loading multiple associations.

Bidirectional relationships require Jackson annotations (@JsonManagedReference/@JsonBackReference) to prevent circular serialization.

2.4 Web Application

2.4.1 Design and its Motivations

The website was built using Handlebars to manage templates, enabling dynamic and well-structured page rendering. For backend communication, the Axios library is employed, allowing efficient and asynchronous API requests. Data is organized into clearly defined sections to provide intuitive navigation and a user-friendly browsing experience.

2.4.2 Issues

During development, some pages experienced performance issues due to the excessive number of elements rendered at once. In particular, the character search page was initially slow because it loaded and displayed too much data simultaneously. This issue was resolved by optimizing data retrieval and limiting the information shown.

Additionally, some pages still contain a large amount of content, such as many characters or anime entries. While functionally correct, this can feel overwhelming or slightly frustrating for users, especially on smaller screens.

2.4.3 Requirements

The web application provides a clear and intuitive user interface that avoid confusing views and uses appropriate visual elements to organize information effectively. Client–server communication is handled through asynchronous JavaScript using Axios and the async/await pattern, ensuring non-blocking and responsive interactions.

The application communicates correctly with the Node.js server through HTTP endpoints, following a consistent request–response structure. Data is correctly retrieved from the backend and rendered in the user interface, while error cases are handled gracefully using appropriate HTTP status codes and fallback views.

2.4.4 Limitations

The web application is mainly designed for desktop use and is not fully optimized for small screens, where pages with many elements may feel crowded.

3. Conclusions

The separation of servers into distinct components enables optimal management of both static and dynamic data, though it necessitates careful oversight of server communication and workload distribution. Combining Express and Spring Boot provides independent and specialized data handling capabilities, yet may introduce challenges regarding scalability and data consistency.

4. Division of Work

The project has been developed collaboratively, with efforts made to maintain an equitable distribution of responsibilities throughout simultaneous work sessions. Consequently, we believe the contribution breakdown is as follows:

- Daniel Luzardo: 33%
- Juan Gonzalez: 33%
- Marta Ibáñez: 33%

5. Extra Information

Running the Application

To run the complete system, the following servers must be started in order:

1. **PostgreSQL Database**: Ensure PostgreSQL is running on port 5432
2. **MongoDB Database**: Ensure MongoDB is running on port 27017
3. **Spring Boot Server** (port 8082):
    1. cd solution/springboot-server 2. ./gradlew bootRun
4. **MongoDB Express Server** (port 3000):
    1. cd solution/mongo-server 2. npm install 3. npm start
5. **Main Express Server** (port 3001):
    1. cd solution/main-server 2. npm install 3. npm start

- **MongoDB Server Swagger Documentation**: http://localhost:3000/api-docs

Database Configuration

The database connection settings can be found in:

- Spring Boot: springboot-server/src/main/resources/application.properties
- MongoDB: mongo-server/app.js

Data Import

The CSV data files should be placed in their respective server directories. Import scripts for MongoDB collections are located in mongo-server/scripts/.

6. Bibliography

Our development process drew upon multiple resources, including course materials from the Moodle platform and documentation from Bootstrap, W3Schools, and JavaDoc. Additionally, we employed ChatGPT as a code generation tool. When incorporating AI-generated solutions, we followed a systematic approach: first analyzing and understanding the provided code, then verifying its accuracy and functionality, and ultimately modifying it to align with our project's specific needs. This methodology ensured that we maintained a strong learning-oriented approach throughout the implementation.